FINAL PROJECT REPORT

Hospital Management System

A Console-Based Patient Record and Billing Application in C++

Submitted by	Syeda Maryyam
Roll / Registration No.	79774
Course	Programming fundementals
Instructor	Ma’am Neelum
Date	April 2026

Language: C++ | IDE: Microsoft Visual Studio | Storage: File Handling (patients.txt)

Table of Contents

Abstract
1. Introduction
2. Problem Statement
3. Project Objectives
4. Scope and Users
5. Tools and Technologies
6. System Features
7. Program Workflow
8. Data Structures and File Handling
9. Compilation and Usage
10. Testing Checklist
11. Limitations and Future Enhancements
12. Conclusion
Appendix A: Screenshots
Appendix B: Function Reference

Abstract

The Hospital Management System is a console-based application developed using C++. It is designed to simplify basic hospital record management by providing separate login options for administrators, doctors, and receptionists.

The system allows users to register, edit, delete, search, and view patient records. It also provides diagnosis management and billing functionality. Patient information is stored using file handling, allowing records to be retrieved when the program is restarted.

1. Introduction

Hospitals and clinics handle a large amount of patient information every day, including personal details, diagnoses, and billing records. When this information is maintained manually on paper, it is slow to search, easy to misplace, and prone to errors.

This project presents a small, console-based Hospital Management System (HMS) written in C++. The program applies core programming concepts, namely structures, arrays, functions, decision-making, loops, input validation, and file handling, to a realistic problem. Access to the system is divided by role, so that each type of staff member sees only the operations relevant to their work.

The program stores up to 100 patient records in memory and saves them to a text file after every change, so that no data is lost when the application is closed.

2. Problem Statement

Small clinics and training environments often lack a simple tool for keeping track of patients. Manual record keeping leads to several problems:

- Records are difficult to search and update quickly.
- Duplicate or inconsistent patient entries can occur.
- Billing is calculated by hand and is prone to arithmetic mistakes.
- All staff may have access to every record, regardless of their role.
- Data is lost easily if it is not stored permanently.

There is therefore a need for a simple application that manages patient records, restricts functions by role, calculates bills automatically, and stores data permanently.

3. Project Objectives

1. To develop a basic hospital management application using C++.
2. To implement a role-based login system.
3. To manage patient information efficiently.
4. To provide patient searching and record modification.

5. To implement a billing system.
6. To use file handling for permanent data storage.
7. To apply fundamental programming concepts to a practical problem.

4. Scope and Users

The system is intended for small-scale use, such as a classroom project, a demonstration, or a very small clinic. It runs in a single console window on one computer and supports one logged-in user at a time.

4.1 User Roles

Administrator: has full access. Can register, edit, delete, search, and view patients, update diagnoses, and generate bills.

Doctor: can search patients, view all patients, and add or update a patient's disease/diagnosis.

Receptionist: can register, edit, delete, search, and view patients, and generate bills. Cannot change diagnoses.

4.2 Role Permissions

Function	Admin	Doctor	Receptionist
Add / Register Patient	Yes	No	Yes
Edit Patient	Yes	No	Yes
Delete Patient	Yes	No	Yes
Search Patient	Yes	Yes	Yes
View All Patients	Yes	Yes	Yes
Add / Update Disease	Yes	Yes	No
Generate Bill	Yes	No	Yes

5. Tools and Technologies

Technology	Purpose
C++	Programming language
Microsoft Visual Studio	Development environment
Structures	Store patient information
Arrays	Maintain patient records
Functions	Divide program into modules
If-else and switch	Decision-making

Technology	Purpose
Loops	Repeated operations
fstream	File handling

The program also uses the standard headers <iostream> (input/output), <cstring> (string operations), <limits> (input clean-up), <iomanip> (setw for safe login input), and <cstdlib> (screen clearing).

6. System Features

6.1 Role-Based Login

The main menu offers Admin, Doctor, and Receptionist login. After the username and password are verified, the matching menu is displayed. Each menu contains only the operations permitted for that role, and a Logout option returns to the main menu.

6.2 Patient Registration

A new patient is registered with a unique positive ID, a name, and an age between 1 and 120. The program rejects duplicate IDs, empty names, and invalid ages. New patients start with the diagnosis "Not Diagnosed" and a bill of zero.

6.3 Edit Patient

The patient is located by ID, the current details are displayed, and the user can enter a new name and age. The changes are saved to the file immediately.

6.4 Delete Patient

After the record is displayed, the user must confirm the deletion with Y or N. When confirmed, the remaining records are shifted up in the array and the file is updated.

6.5 Search Patient

Searching is done by patient ID. If a matching record exists, all of its details, including charges and total bill, are shown.

6.6 View All Patients

Displays every registered patient together with the total number of records. A message is shown if no records exist.

6.7 Disease / Diagnosis Management

Doctors and administrators can add or update the disease or diagnosis for a patient. Empty entries are not accepted.

6.8 Billing

The user enters the consultation fee, medicine charges, and other charges. The program rejects negative values, calculates the total bill automatically (consultation fee + medicine charges + other charges), prints a formatted hospital bill, and saves the result.

6.9 Input Validation

The helper functions readInt() and readText() repeatedly prompt the user until valid input is entered. Non-numeric input, empty text, and text that is too long are all handled without crashing the program.

6.10 Permanent Storage

Records are written to patients.txt after each change and when the program exits. They are loaded automatically at start-up.

7. Program Workflow

The program follows the sequence below each time it is run:

1. The program starts and loadFromFile() reads any saved records from patients.txt into the patients array.
2. The main menu is shown with four options: Admin Login, Doctor Login, Receptionist Login, and Exit.
3. The user selects a role and enters a username and password. If they are incorrect, the user is returned to the main menu.
4. On success, the role-specific menu (adminMenu, doctorMenu, or receptionistMenu) is displayed.
5. The user selects an operation. The corresponding function runs, validates input, updates the records, and (for changes) calls saveToFile().
6. After each operation the user presses Enter and the role menu is shown again.
7. Choosing Logout returns to the main menu. Choosing Exit saves all records and ends the program.

7.1 Menu Structure

Main Menu Option	Opens	Menu Items
1. Admin Login	Admin Menu	Add, Edit, Delete, Search, View All, Add/Update Disease, Generate Bill, Logout
2. Doctor Login	Doctor Menu	Search, View All, Add/Update Disease, Logout
3. Receptionist Login	Receptionist Menu	Register, Edit, Delete, Search, View All, Generate Bill, Logout

Main Menu Option	Opens	Menu Items
4. Exit	-	Saves records and closes the program

8. Data Structures and File Handling

8.1 Patient Structure

All patient information is held in a single structure:

```
struct Patient {
    int  id;
    char name[50];
    int  age;
    char disease[50];
    int  consultationFee;
    int  medicineCharges;
    int  otherCharges;
    int  bill;
};
```

Field	Type	Description
id	int	Unique positive patient identifier
name	char[50]	Patient's name
age	int	Age in years (1 to 120)
disease	char[50]	Diagnosis (default: Not Diagnosed)
consultationFee	int	Doctor's consultation charge
medicineCharges	int	Cost of medicines
otherCharges	int	Any additional charges
bill	int	Total of the three charges above

8.2 Arrays and Constants

Patients are stored in a global array, Patient patients[MAX_PATIENTS], where MAX_PATIENTS is 100. The variable totalPatients records how many entries are in use. The function findPatient() performs a linear search by ID and returns the array index, or -1 if the patient does not exist.

8.3 File Handling

Data is saved in a plain text file named patients.txt using ofstream and loaded with ifstream. Each patient occupies eight lines, in the following order: ID, name, age, disease, consultation fee, medicine charges, other charges, and total bill. For example:

```
101
Ali Khan
35
Fever
1500
800
200
2500
```

- **Saving:** saveToFile() rewrites the whole file from the array and reports success or failure.
- **Loading:** loadFromFile() reads records until the end of file or until 100 records are loaded. Invalid records (non-positive ID, age outside 1-120, or negative charges) and duplicate IDs are skipped, which protects the program from a corrupted data file.
- **First run:** If the file does not exist, the program starts with no records and creates the file on the first save.

9. Compilation and Usage

9.1 Building the Project

1. Open Microsoft Visual Studio and create a new C++ Console App project.
2. Add the source file (for example, main.cpp) containing the program code.
3. Build the solution (Ctrl + Shift + B) and run it (Ctrl + F5).
4. The file patients.txt is created in the project's working directory the first time a record is saved.

Note: the code uses system("cls") and strcpy_s(), which are specific to Windows and Microsoft's compiler. See Section 11 for portability notes.

9.2 Default Login Credentials

Role	Username	Password
Administrator	`admin`	`123`
Doctor	`doctor`	`123`
Receptionist	`reception`	`123`

These credentials are hard-coded for demonstration purposes and should be changed before any real use.

10. Testing Checklist

The following test cases cover the main behaviour of the program. Mark the Result column after running each test.

No.	Test Case	Expected Result	Result
1	Login with correct credentials	Login successful message; role menu is displayed	[ ]
2	Login with wrong username or password	Error message; returns to main menu	[ ]
3	Enter letters instead of a menu number	"Invalid input" message; prompt repeats	[ ]
4	Enter an invalid menu number	"Invalid choice" message	[ ]
5	Register a patient with a new ID	Patient registered and record saved	[ ]
6	Register a patient with an existing ID	"This patient ID already exists"	[ ]
7	Register with ID of 0 or a negative number	"Patient ID must be positive"	[ ]
8	Enter an age of 0 or above 120	Prompt for an age between 1 and 120	[ ]
9	Leave the name empty	"This field cannot be empty"	[ ]
10	Edit an existing patient	Name and age updated; record saved	[ ]
11	Edit a patient that does not exist	"Patient not found"	[ ]
12	Delete a patient and confirm with Y	Record removed and file updated	[ ]
13	Delete a patient and answer N	"Deletion cancelled"; record kept	[ ]
14	Search for an existing / missing ID	Record shown / "Patient not found"	[ ]
15	View all patients with no records	"No patient records available"	[ ]
16	View all patients with records	All records listed with total count	[ ]
17	Add or update a disease	Diagnosis updated and saved	[ ]
18	Generate a bill with valid charges	Total equals the sum of the three charges	[ ]
19	Generate a bill with a negative charge	Error shown; bill not changed	[ ]
20	Close and restart the program	Saved records are loaded from patients.txt	[ ]
21	Log in as Doctor	Only 4 options available; no billing or deletion	[ ]
22	Log in as Receptionist	No disease update option available	[ ]
23	Register a 101st patient	"Patient storage is full"	[ ]

11. Limitations and Future Enhancements

11.1 Current Limitations

- The system holds a maximum of 100 patients because it uses a fixed-size array.
- Usernames and passwords are hard-coded in the source code and are not encrypted.
- Patient data is saved as plain, unencrypted text.
- Patients can be searched only by ID, not by name or disease.
- Editing changes only the name and age.
- The program uses system("cls") and strcpy_s(), so it builds only on Windows with Visual Studio unless these are replaced.
- There is no appointment scheduling, billing history, or payment status.
- It is a console application and supports a single user at a time.

11.2 Future Enhancements

- Appointment scheduling for doctors and patients.
- Database integration (for example, SQLite or MySQL) instead of a text file.
- A graphical user interface (GUI).
- Secure authentication with password hashing and user management.
- Dynamic storage using std::vector and std::string to remove the 100-record limit.
- Cross-platform support by replacing Windows-specific calls.
- Search by name, report generation, and printable invoices.
- Medical history and doctor assignment for each patient.

12. Conclusion

The Hospital Management System demonstrates the practical application of fundamental C++ programming concepts in developing a basic management application. It provides essential patient-management operations, diagnosis updates, billing facilities, and file-based data storage.

The project helped develop an understanding of structured programming, functions, arrays, input validation, and file handling. It also provides opportunities for future improvements, including appointment scheduling, database integration, and a graphical user interface.

Appendix A: Screenshots

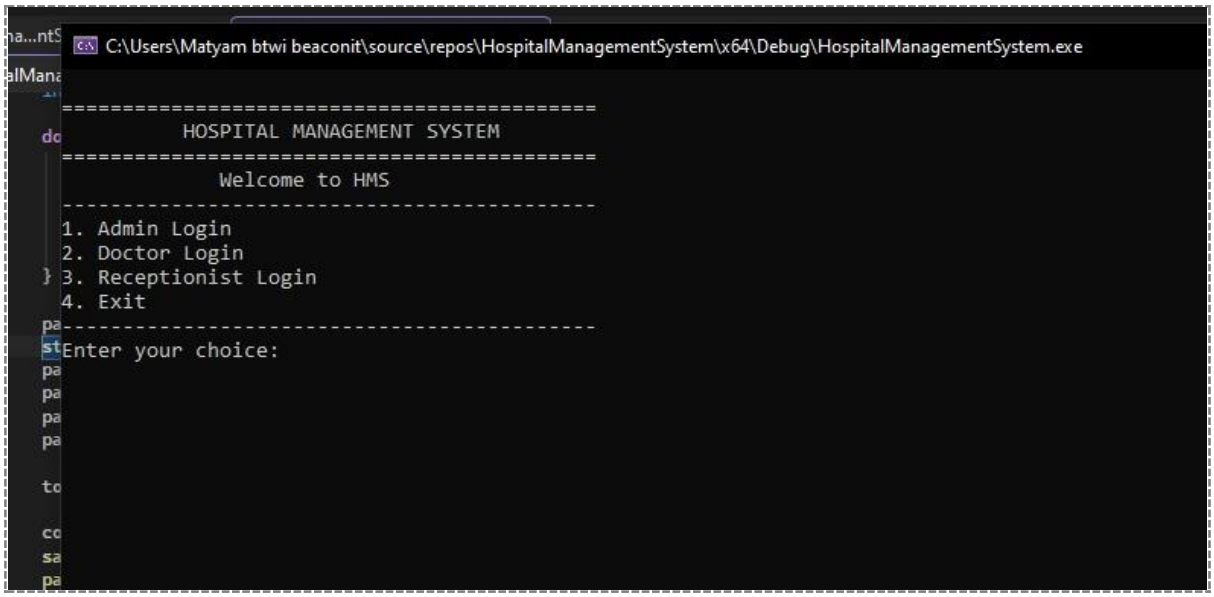


Figure A.1: Main menu

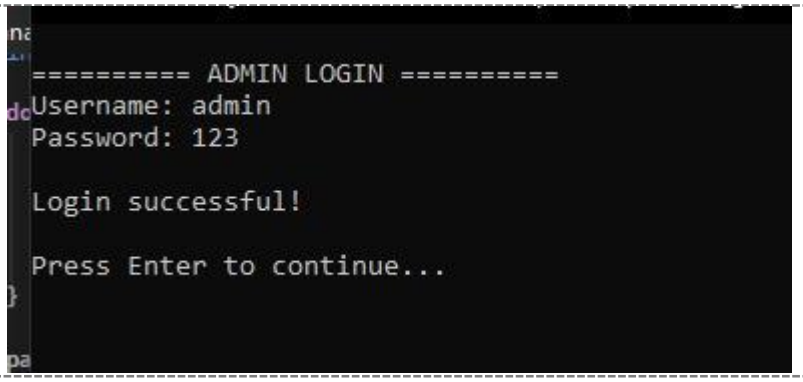


Figure A.2: Admin login

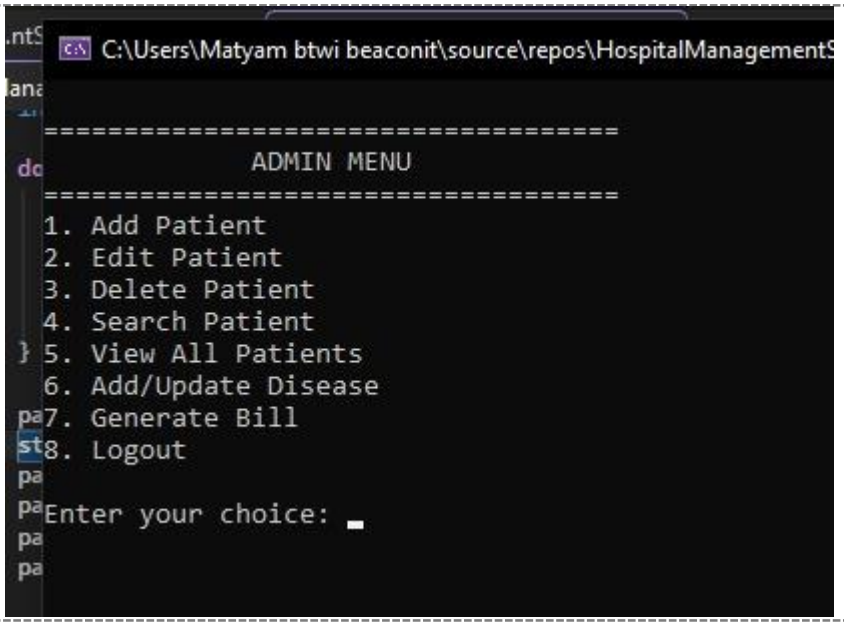


Figure A.3: Admin menu

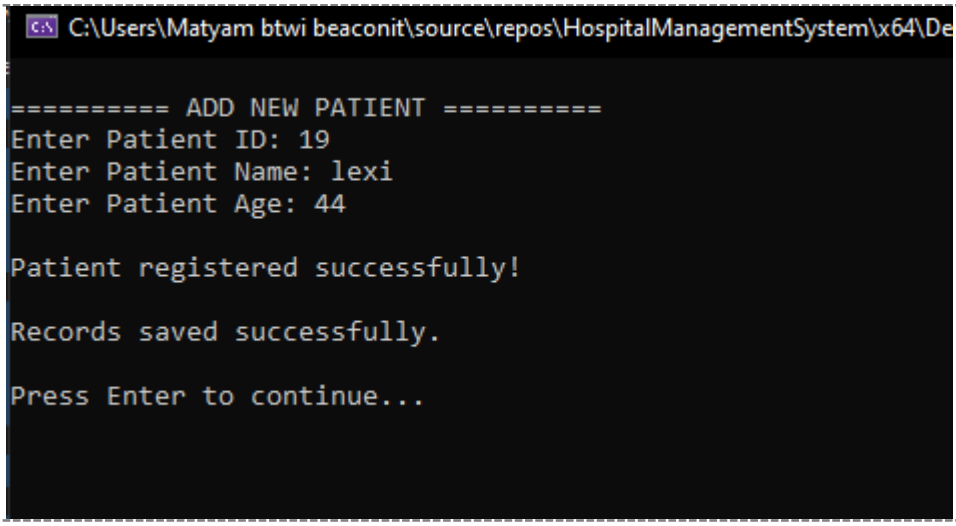


Figure A.4: Register a new patient

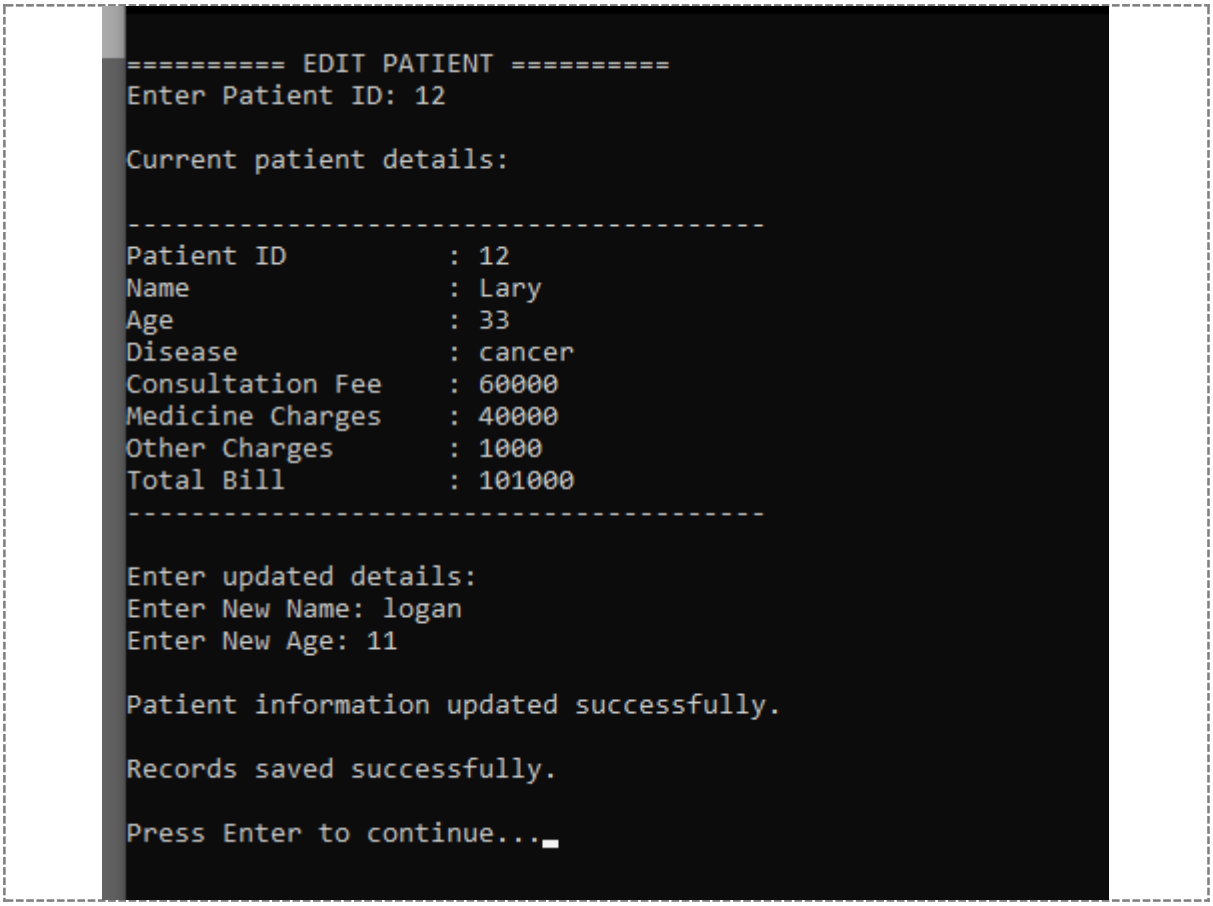


Figure A.5: Edit patient

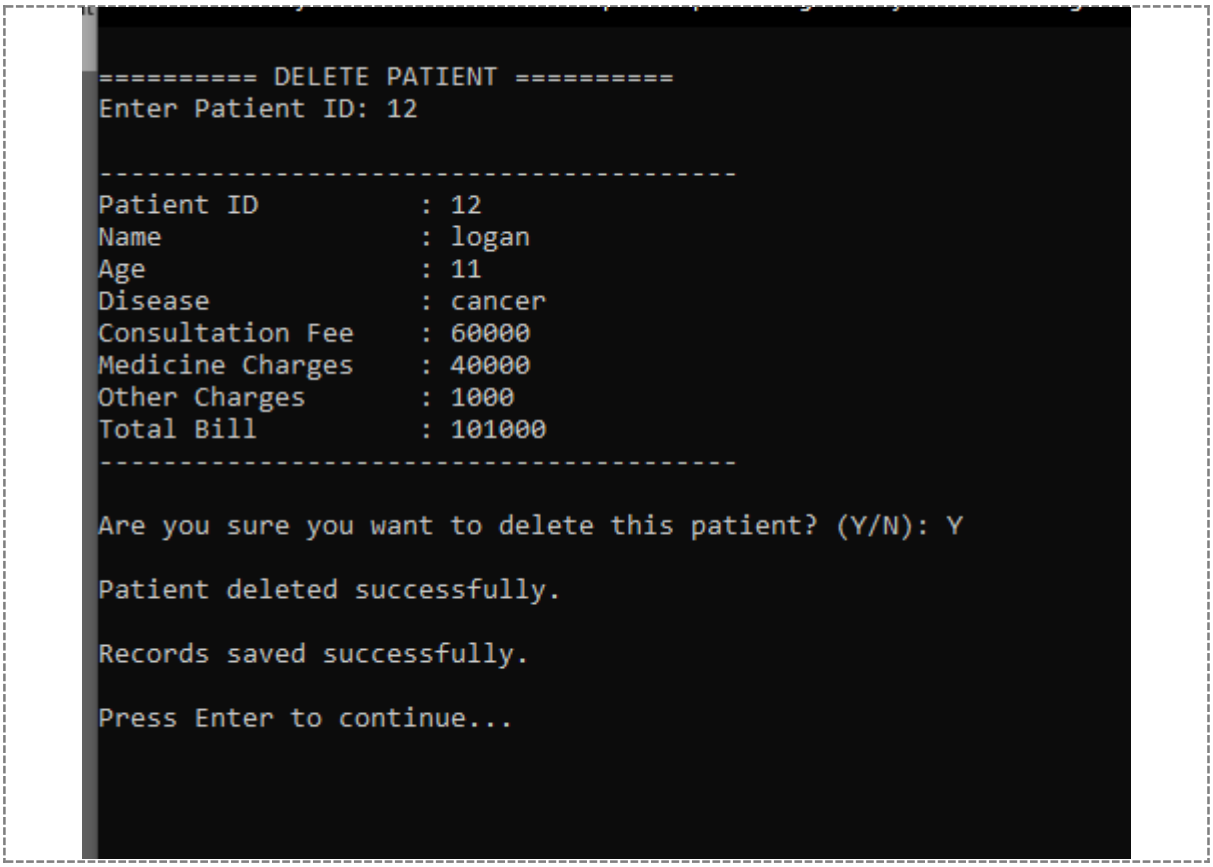


Figure A.6: Delete patient

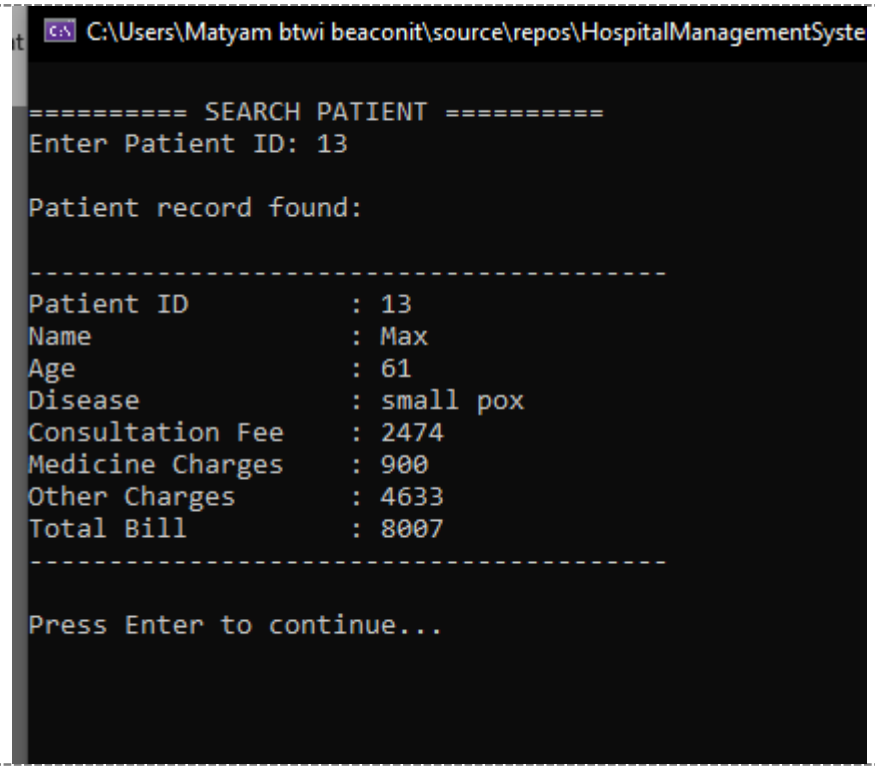


Figure A.7: Search patient

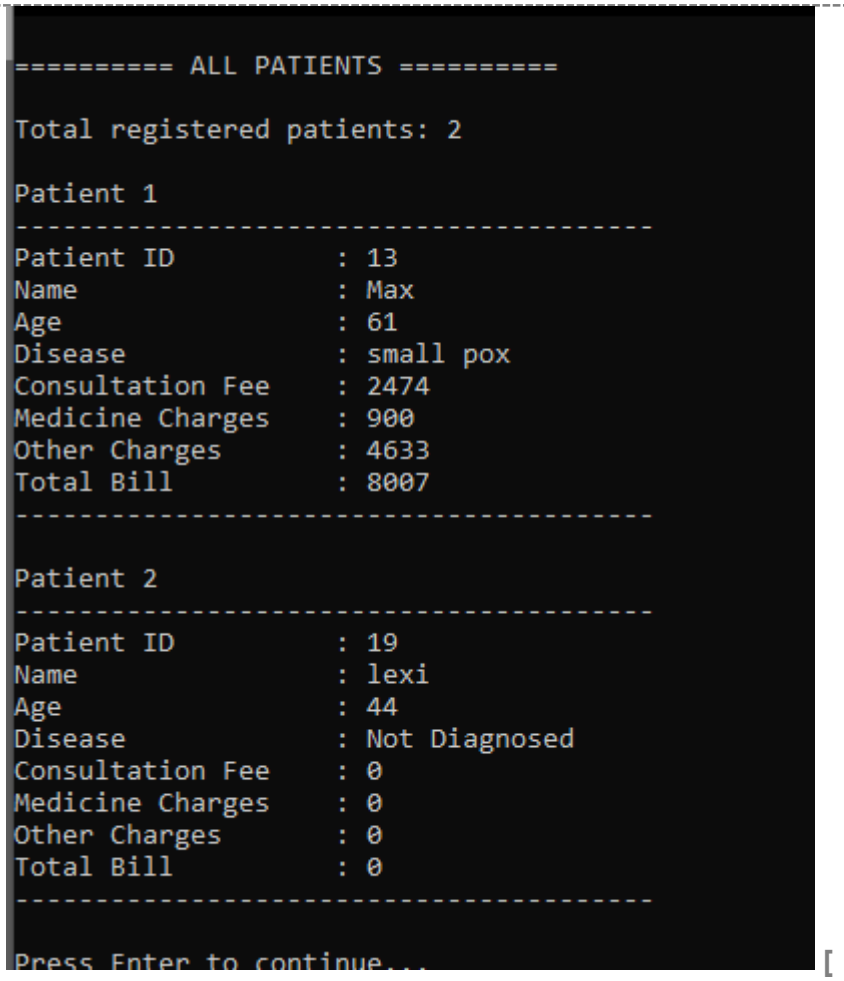


Figure A.8: View all patients

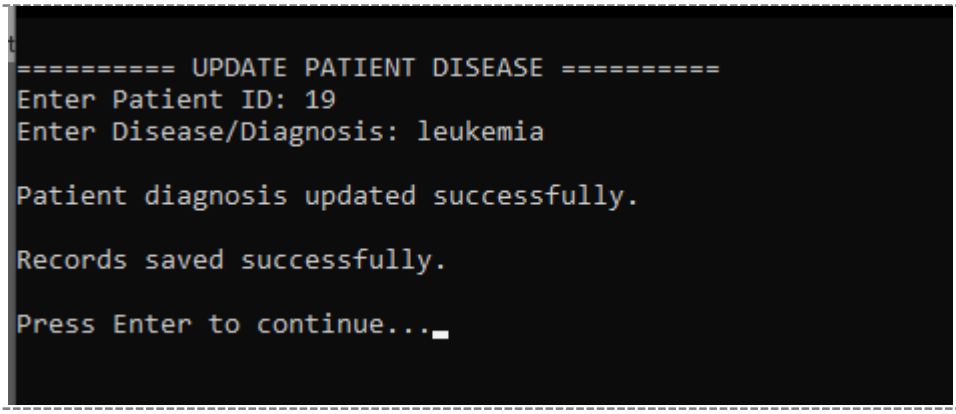


Figure A.9: Update disease

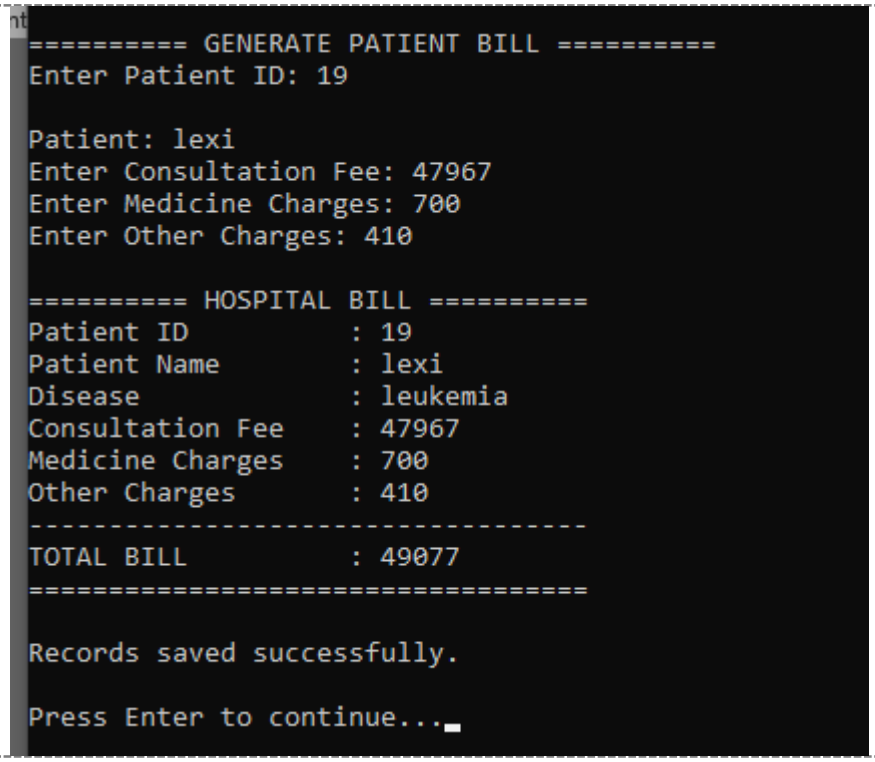


Figure A.10: Generate bill

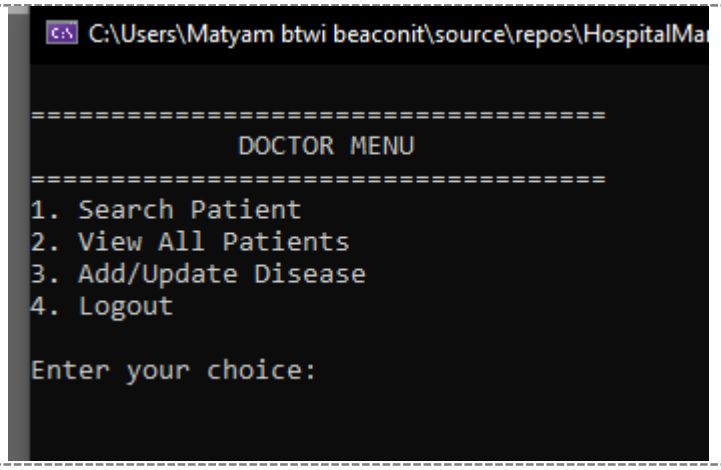


Figure A.11: Doctor menu

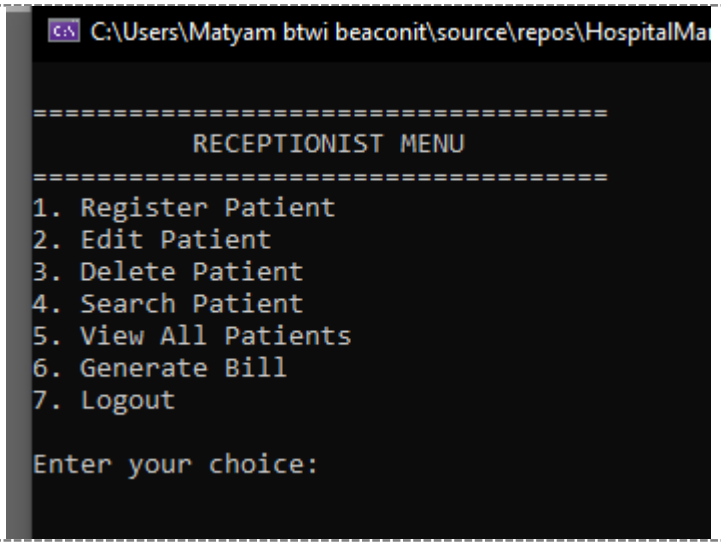


Figure A.12: Receptionist menu

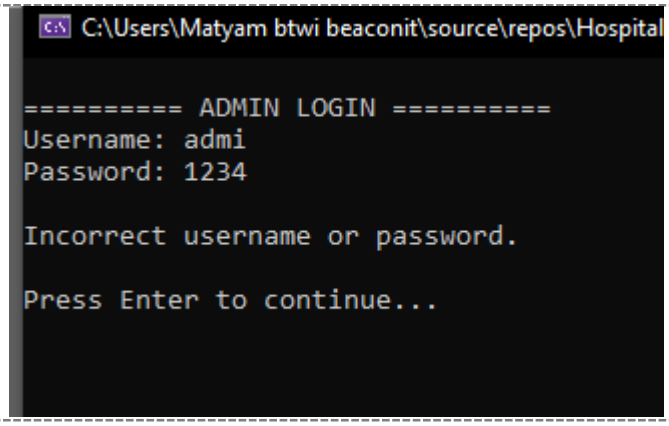


Figure A.13: Input validation

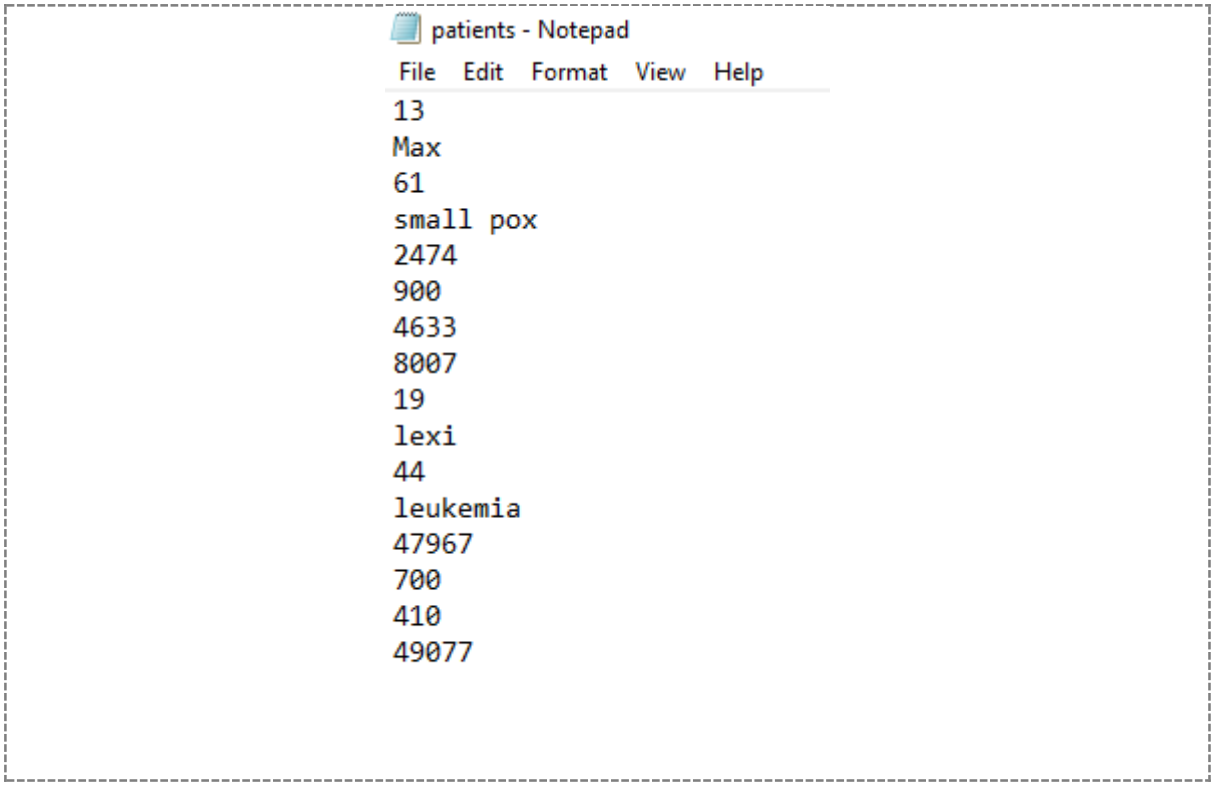


Figure A.14: Stored data file

Appendix B: Function Reference

Function	Description
`readInt()`	Prompts for a whole number and repeats until valid input is entered
`readText()`	Reads a non-empty line of text with length checking
`pauseScreen()`	Waits for the user to press Enter
`clearScreen()`	Clears the console window
`findPatient()`	Searches the array by ID; returns the index or -1
`calculateBill()`	Sets the bill as consultation + medicine + other charges
`saveToFile()`	Writes all records to patients.txt
`loadFromFile()`	Reads and validates records from patients.txt at start-up
`displayPatient()`	Prints the full details of one patient
`addPatient()`	Registers a new patient after validating ID, name, and age
`editPatient()`	Updates the name and age of an existing patient
`deletePatient()`	Removes a patient after confirmation
`searchPatient()`	Finds and displays a patient by ID
`viewPatients()`	Lists all registered patients
`addDisease()`	Adds or updates a patient's diagnosis
`generateBill()`	Collects charges, calculates the total, and prints the bill
`login()`	Verifies the username and password for a role
`adminMenu()`	Displays and handles the administrator menu
`doctorMenu()`	Displays and handles the doctor menu
`receptionistMenu()`	Displays and handles the receptionist menu
`main()`	Loads data, shows the main menu, and routes logins